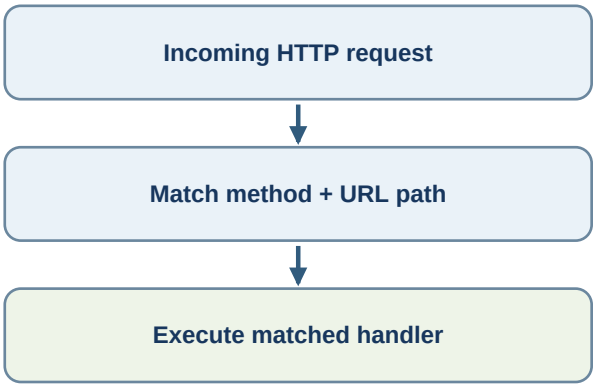
6. WHAT IS ROUTING IN BACKEND?

How Requests Find Their Way Home



Handler runs validation, business logic and required data operations.

Core idea: HTTP methods describe WHAT action the client wants to perform. Routing describes WHERE that action should be applied.

1. What is Routing?

Routing is the process of mapping an incoming HTTP request—typically using its HTTP method and URL path—to a specific server-side **handler**.

A **handler** is a function, controller or set of instructions that processes the request. It may validate input, check authentication and authorization, call business logic, interact with a database and create an HTTP response.

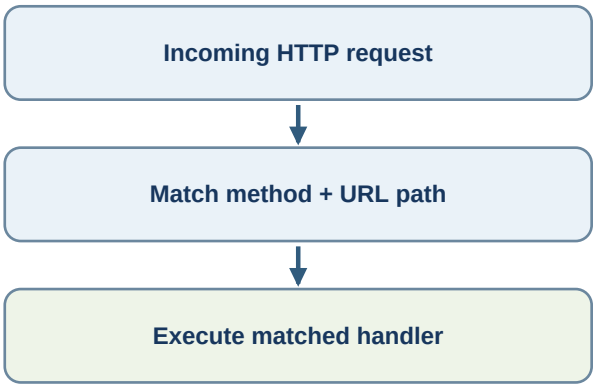
The WHAT vs WHERE distinction

Element	Meaning	Example
HTTP method	WHAT action is intended	GET = retrieve
URL path	WHERE / which resource	/api/books
Route	Method + path matching rule	GET /api/books
Handler	Code that runs after a match	listBooksHandler

For example, GET /api/books and POST /api/books share the same path but express different intentions. They can therefore map to different handlers.

How does the server distinguish routes?

A useful conceptual model is that the method and path together identify a route. The routing framework matches the incoming request against registered route definitions. It is not necessary for every server to literally concatenate the strings; the important idea is that method and path are considered together.



Handler runs validation, business logic and required data operations.

2. Types of Routes

There are two common ways to structure route paths: static and dynamic.

A. Static Routes

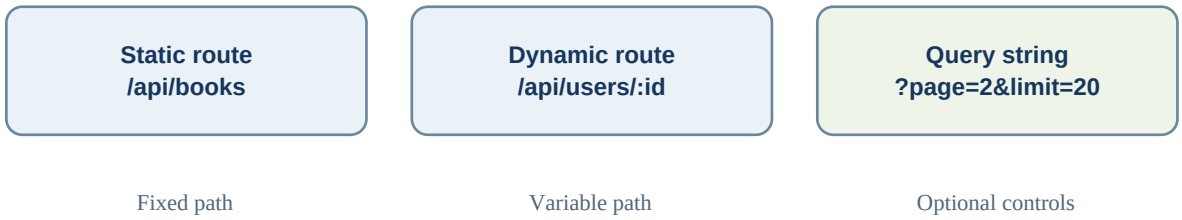
A **static route** has a fixed path. It does not contain variable path parameters. The path remains the same for every request that matches it.

```
GET  /api/books
POST /api/books
GET  /api/health
```

Common uses include fetching a collection of books, creating a book and checking whether the server is healthy. Different HTTP methods can still map to different handlers on the same static path.

B. Dynamic Routes

A **dynamic route** contains one or more variable slots in its path. The router extracts values from the actual request URL.




```
Route pattern:  /api/users/:id
Actual request: GET /api/users/123
Extracted value: id = "123"
```

The colon syntax (:id) is common in frameworks such as Express. Other frameworks use different syntax; for example, FastAPI uses {user_id}.

3. Path Parameters vs Query Parameters

Backend engineers commonly use two kinds of URL parameters to pass information to an endpoint.

A. Path Parameters (Route Parameters)

A **path parameter** is a variable value placed directly inside the URL path, usually after a forward slash. It commonly identifies a specific resource.

```
GET /api/users/123

123 is the path parameter.
It identifies the user whose ID is 123.
```

Other examples: /api/products/45, /api/orders/ORD-9001 and /api/teams/7.

B. Query Parameters

A **query parameter** is a key-value pair placed at the end of a URL after a question mark (?). Multiple parameters are separated by an ampersand (&).

```
GET /api/search?query=backend&sort=latest

Path: /api/search
query = backend
sort  = latest
```

Common uses of query parameters

- **Pagination:** page=2&limit;=20
- **Filtering:** category=books
- **Sorting:** sort=price■=asc
- **Searching:** query=backend

Query parameters are especially common for GET requests because GET requests generally use the URL to express retrieval options rather than relying on a request body. HTTP does not categorically prohibit a GET body, but its semantics are not generally defined and many systems do not support it consistently.

Path parameter	Query parameter
Part of the URL path.	Appears after ? in the URL.
Commonly identifies a resource.	Commonly filters, sorts or configures results.
/users/123	/users?role=admin

Rule of thumb: Path parameters answer WHICH resource? Query parameters answer WHICH subset or HOW should the results be returned?

4. Nested Routing

Nested routing is a common REST API design practice used to express a hierarchy or relationship between resources.

```
GET /api/users/123/posts/456
```

Example sequence

```
/api/users
Fetches a collection of users.

/api/users/123
Identifies a specific user.

/api/users/123/posts
Fetches posts belonging to user 123.

/api/users/123/posts/456
Identifies post 456 in the context of user 123.
```

Nested paths make relationships readable and help express what data the client wants. Avoid excessive nesting: use only the levels that make the resource relationship clearer.

5. Route Versioning and Deprecation

As applications grow, business requirements change. An API may need to change its response format, endpoint behaviour or data contract.

The problem: Breaking changes

Suppose an existing API returns a field called name, but a new requirement changes that field to title. A frontend application that expects name may stop working if the backend changes the existing response without warning.

```
Version 1:
{ "id": 1, "name": "Laptop" }

Version 2:
{ "id": 1, "title": "Laptop" }
```

The solution: API Versioning

API versioning means maintaining different versions of an API contract so clients can migrate without being forced to update immediately.

```
GET /api/v1/products
GET /api/v2/products
```

- v1 continues returning the old response structure.
- v2 returns the updated response structure.
- Frontend teams migrate their applications to v2.
- After the migration window and usage review, v1 can be retired.

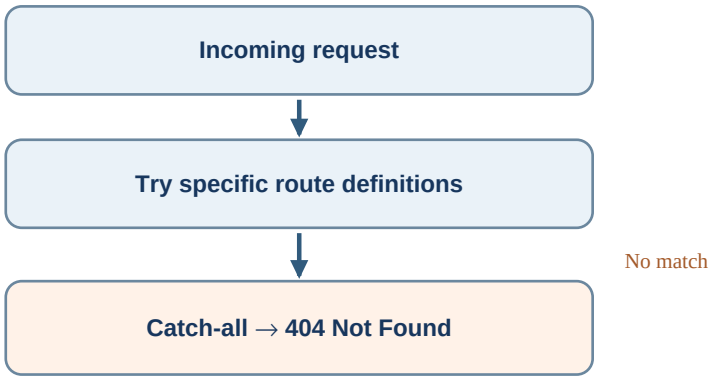
Deprecation

Deprecation is the process of announcing that an API version or feature is no longer recommended and is planned for removal in the future. Deprecation is not the same as immediate removal.

- Release the new API version.
- Announce that the old version is deprecated.
- Document the differences and migration steps.
- Give clients a reasonable period to migrate.
- Monitor usage and communicate a removal date.
- Retire the old version after the transition period, where appropriate.

6. Catch-All Routes

A **catch-all route** is a fallback route that handles requests that do not match any previously registered route. It is generally placed at the end of the routing configuration so valid, specific routes get a chance to match first.



A catch-all route may use wildcard syntax, but exact syntax depends on the backend framework and its version.

Why is it useful?

Without deliberate fallback handling, an invalid endpoint may produce an unclear framework default response. A catch-all handler can return a consistent, user-friendly error.

```
{
  "error": "Not Found",
  "message": "The requested endpoint does not exist."
}
```

HTTP 404 Not Found indicates that the requested resource or route could not be found. Register the fallback after specific routes so it does not intercept valid requests.

7. Code Examples

Express.js (Node.js)

```
import express from "express";

const app = express();

// Static route
app.get("/api/books", (req, res) => {
  res.json({ message: "List of books" });
});

// Dynamic route: path parameter
app.get("/api/users/:id", (req, res) => {
  const userId = req.params.id;
  res.json({ userId });
});

// Query parameters
app.get("/api/search", (req, res) => {
  const query = req.query.query;
  const sort = req.query.sort;
  res.json({ query, sort });
});

// Catch-all / not-found handler
app.use((req, res) => {
  res.status(404).json({ error: "Route not found" });
});

app.listen(3000);
```

FastAPI (Python)

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/api/books")
def list_books():
    return {"message": "List of books"}

@app.get("/api/users/{user_id}")
def get_user(user_id: int):
    return {"user_id": user_id}

@app.get("/api/search")
def search(query: str, page: int = 1, limit: int = 20):
    return {
        "query": query,
        "page": page,
        "limit": limit
    }
```

8. Common Routing Mistakes

- **Confusing path and query parameters:** /users/123 identifies a user; /users?role=admin filters users.
- **Ignoring validation:** Validate path and query values before using them.
- **Overly deep nesting:** Very long paths can become difficult to maintain.
- **Missing 404 handling:** Unknown endpoints should receive a clear not-found response.
- **Breaking existing clients:** Use versioning and a migration plan for breaking API changes.
- **Ambiguous route patterns:** Understand route precedence in the framework.
- **Putting all logic in the router:** Keep route definitions readable and move substantial business logic into service/controller layers.

9. Quick Revision Sheet

Concept	Definition
Routing	Maps an incoming request to a server-side handler.
HTTP method	Describes the intended action (WHAT).
URL path	Identifies the endpoint/resource (WHERE).
Route handler	Function/controller that processes a matched request.
Static route	Fixed path without variable path segments.
Dynamic route	Path containing variable segments.
Path parameter	Variable embedded in the path, often identifying a resource.
Query parameter	Key-value input after ?, often used for filtering or sorting.
Nested route	Path expressing a relationship between resources.
API versioning	Maintaining multiple API contracts such as v1 and v2.
Deprecation	Announcing that a version/feature will be retired later.
Catch-all route	Fallback for requests not matched by specific routes.
404 Not Found	Status code for a route/resource that cannot be found.

Most important takeaway:
HTTP method = WHAT action?
URL path = WHERE / which resource?
Path parameter = WHICH specific resource?
Query parameter = WHICH subset / HOW to retrieve it?
Router = WHICH handler should execute?

10. Backend Interview Questions

1. What is routing in backend development?

Routing maps an incoming request, usually using its HTTP method and URL path, to the handler responsible for processing it.

2. What is the difference between a route and a route handler?

A route is the matching rule (method + path); a handler is the function that executes when the rule matches.

3. What is the difference between static and dynamic routes?

A static route has a fixed path such as /api/books; a dynamic route has variable path segments such as /api/users/:id.

4. What is a path parameter?

A variable embedded in the URL path, commonly used to identify a resource, such as 123 in /api/users/123.

5. What is a query parameter?

A key-value input after ?, commonly used for filtering, sorting, searching and pagination.

6. Path parameter vs query parameter?

Path parameters commonly identify a resource; query parameters commonly modify how a collection is filtered or returned.

7. Why use nested routes?

They express relationships between resources, such as the posts belonging to a particular user.

8. Why is API versioning needed?

It allows breaking changes while existing clients continue using a compatible older contract during migration.

9. What does deprecation mean?

A version or feature is marked as no longer recommended and scheduled for removal after a transition period.

10. What is a catch-all route?

A fallback route that handles requests not matched by earlier route definitions, often returning 404.

11. Why should a catch-all route be registered last?

So it does not intercept requests that should match more specific routes.

12. Does the router itself perform database operations?

Not necessarily. It selects a handler; the handler or service layer performs business logic and database operations.

Source Video

6. What is Routing in Backend? How Requests Find Their Way Home